چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۱۱

لایهٔ شبکه — بخش ۱: HTTP، Retrofit، OkHttp، Ktor و فراخوانی‌های مقاوم



اسماعیل مرادی

@esmaeeil-moradi700

چرا لایهٔ شبکه در ۲۰۲۶ مهم است؟

تقریباً هر صفحه‌ای منتظر یک سرور است — و شبکهٔ موبایل کند و ناپایدار است و گاهی دروغ می‌گوید («وصل است» ولی اینترنت ندارد). مصاحبه‌گرها با این موضوع می‌سنجند که آیا برای **شکست** طراحی می‌کنید، نه فقط برای مسیر خوش‌بینانه:

- **Retrofit و OkHttp** هنوز انتخاب پیش‌فرض اندروید است؛ **کلاینت Ktor** انتخاب KMP است
- در کد جدید Kotlin، **kotlinx.serialization** جای Gson را گرفته است
- فراخوانی API با `suspend` : بدون call adapter، main-safe و قابل لغو
- مقاوم بودن یک کار طراحی است: timeout، retry با backoff، خطاهای تایپ‌دار
- HTTP caching و وضعیت اتصال به **درستی** برنامه مربوط‌اند، نه فقط به سرعت

سطح	سؤال‌ها
جونیور	مبانی HTTP، ترکیب Retrofit و OkHttp، serialization
مید	interceptorها، timeout و retry، مدل‌سازی خطا، HTTP caching
سینیور	Ktor و KMP، وضعیت اتصال، لغو و هم‌روندی
آرکیتکت	طراحی ماژول شبکه، مقاومت در مقیاس بالا، تکامل API



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: مسیر یک درخواست

```
Repository     suspend fun user(id): User
    |
    ▼
Retrofit/Ktor  Kotlin call → HTTP request · JSON → DTO
    |
    ▼
OkHttp         interceptors · cache · redirects · retries
    |
    ▼
Connections    pool · DNS · TLS · HTTP/2
    |
    ▼
Server  ──►  status + headers + body  ──►  back up
```

- **Retrofit** یا **Ktor** API را *توصیف* می‌کنند؛ **OkHttp** (یا یک engine دیگر) آن را *اجرا* می‌کند
- **کد شما** صاحب سیاست است: کدام خطا retry شود، چه چیزی cache شود، کاربر چه ببیند
- DTOها در لایهٔ data می‌مانند — UI هیچ‌وقت مدل شبکه را نمی‌بیند

سؤالی که پشت همهٔ سؤال‌های شبکه است

«اگر این فراخوانی کند باشد، وسط کار شکست بخورد، یا دو بار اجرا شود چه می‌شود؟» جواب این را بدهید، جواب مصاحبه‌گر را داده‌اید.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

مبانی HTTP، ترکیب Retrofit و OkHttp، serialization



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. متدهای HTTP: امن در برابر idempotent

متد	امن (safe)	idempotent	کاربرد رایج
`GET` / `HEAD`	بله	بله	خواندن یک منبع
`POST`	خیر	خیر	ساختن، یا اجرای یک فرمان
`PUT`	خیر	**بله**	جایگزینی کامل یک منبع
`PATCH`	خیر	تضمین‌شده نیست	به‌روزرسانی بخشی از منبع
`DELETE`	خیر	**بله**	حذف یک منبع

- **امن:** کلاینت هیچ تغییر state روی سرور درخواست نمی‌کند
- **idempotent:** فرستادن همان درخواست به تعداد N بار همان *اثری* را دارد که یک بار — ولی *پاسخ* ممکن است فرق کند (دومین `DELETE` می‌تواند `404` برگرداند)

تلهٔ مصاحبه

«*POST و DELETE هیچ‌کدام idempotent نیستند.*» — نیمی از این جمله غلط است، و نسخهٔ پارسال همین چالش هم همین را گفته بود. طبق مشخصات HTTP، `DELETE` idempotent **است**؛ `POST` نیست. و idempotent یعنی «state را تغییر نمی‌دهد» نیست: `PUT` state را تغییر می‌دهد، اما تکرارش چیز بیشتری را عوض نمی‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. status codeهایی که باید به آن‌ها واکنش نشان دهید

کد	معنی	واکنش کلاینت
`200` ، `201` ، `204`	موفق، ساخته شد، بدون محتوا	از body استفاده کنید — `204` body **ندارد**
`400` ، `422`	درخواست نامعتبر	خطای اعتبارسنجی؛ retry نکنید
`401`	احراز هویت نشده	refresh توکن و یک بار تکرار (روز ۱۲)
`403`	هویت معلوم، بدون اجازه	refresh نکنید؛ «دسترسی ندارید»
`408` ، `429`	timeout، درخواست زیاد	بعداً retry؛ طبق `Retry-After`
`500` ، `502` ، `503` ، `504`	خطای سرور یا gateway	retry با backoff — فقط idempotent

تلهٔ مصاحبه

«401 و 403 یک معنی دارند.» — `401` یعنی نمی‌دانیم شما کی هستید (اعتبارنامه نیست یا منقضی شده). `403` یعنی می‌دانیم، و جواب منفی است. refresh توکن روی `403` کمکی نمی‌کند.

سؤال تکمیلی

«چرا HTTP/2 روی موبایل مهم است؟» — درخواست‌ها روی **یک** اتصال مشترک می‌روند (multiplexing)، پس handshake کمتر. OkHttp خودکار آن را مذاکره می‌کند؛ HTTP/3 (همان QUIC) به Cronet نیاز دارد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. Retrofit و OkHttp: چه کسی چه کاری می‌کند

```
interface UserApi {
    @GET("users/{id}")
    suspend fun user(@Path("id") id: String): UserDto

    @POST("users")
    suspend fun create(@Body body: NewUser): Response<UserDto>
}

val contentType = "application/json".toMediaType()
val retrofit = Retrofit.Builder()
    .baseUrl("https://api.example.com/v1/")
    .client(okHttpClient) // one shared instance
    .addConverterFactory(json.asConverterFactory(contentType))
    .build()

val api: UserApi = retrofit.create()
```

- **Retrofit** اینترفیس را به فراخوانی HTTP تبدیل می‌کند و bodyها را convert می‌کند
- **OkHttp** کار اصلی را انجام می‌دهد: connection pool، HTTP/2، TLS، gzip شفاف، cache، redirect و interceptorها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. جزئیاتی که مصاحبه‌گر می‌پرسد

- `suspend fun user(): UserDto` — روی پاسخ غیر 2xx خطای `HttpException` و روی قطعی شبکه `IOException` پرتاب می‌کند
- `suspend fun create(): Response<UserDto>` — برای خطای HTTP استثنایی پرتاب نمی‌کند: `isSuccessful`، `code()` و `errorBody()` را چک کنید. قطعی شبکه همچنان پرتاب می‌شود.
- یک `OkHttpClient` برای کل اپ — هر client، connection pool و thread pool خودش را دارد

تلهٔ مصاحبه

«برای تابع‌های suspend به `CoroutineCallAdapterFactory` نیاز دارید.» — قدیمی است (اسلایدهای پارسال هم همین را گفته بودند). Retrofit از نسخهٔ **2.6** به‌طور بومی از `suspend` پشتیبانی می‌کند. آن adapter برای برگرداندن `Deferred` بود.

تلهٔ مصاحبه

`baseUrl` باید با `/` تمام شود، و اسلش اول در `@GET("/users")` مسیر را از **ریشهٔ host** حساب می‌کند — یعنی بی‌صدا `/v1/` را حذف می‌کند. از مسیر نسبی استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. serialization: چرا kotlinx.serialization

	kotlinx.serialization	Moshi	Gson
روش	پلاگین کامپایلر، بدون reflection	تولید کد با KSP یا reflection	reflection
آشنا با Kotlin	مقدار پیش‌فرض و nullability	بله	**خیر**
KMP	بله	خیر	خیر
وضعیت	پیش‌فرض کد جدید	نگهداری می‌شود	حالت نگهداری (maintenance)

```kotlin
@Serializable
data class UserDto(
    val id: String,
    @SerialName("display_name") val name: String,
    val avatarUrl: String? = null,
)
```

تلهٔ مصاحبه

«*Gson با Kotlin مشکلی ندارد.*» — Gson شیء را **بدون صدا زدن constructor** می‌سازد: مقدارهای پیش‌فرض نادیده گرفته می‌شوند و یک `String` غیر nullable ممکن است `null` شود و جایی دور از کد parse کرش کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. parser را برای دنیای واقعی تنظیم کنید

```
val json = Json {
    ignoreUnknownKeys = true   // server adds a field → no crash
    coerceInputValues = true   // null or unknown enum → default
    explicitNulls = false      // missing nullable field → null
}
```

- `Json` پیش‌فرض روی کلید ناشناخته **exception پرتاب می‌کند** — یک فیلد جدید در بک‌اند می‌تواند همهٔ اپ‌های نصب‌شده را خراب کند
- هر چیزی که سرور *ممکن است* نفرستد را nullable یا با مقدار پیش‌فرض مدل کنید
- DTOها را در مرز لایهٔ data به مدل domain تبدیل کنید (روز ۳)

در لبه PARSE کنید، یک بار اعتبارسنجی کنید

DTO فرمت داده روی سیم را توصیف می‌کند؛ مدل domain قواعد شما را. جدا نگهشان دارید تا تغییر یک اسم در بک‌اند فقط یک mapper را عوض کند، نه بیست صفحه را.

سؤال تکمیلی

«parse کردن روی کدام ترد اجرا می‌شود؟» — converterهای Retrofit روی تردهای پس‌زمینهٔ OkHttp اجرا می‌شوند، پس یک پاسخ بزرگ ترد اصلی را قفل نمی‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

میدل

interceptorها، timeout و retry، مدل‌سازی خطا، HTTP caching



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. زنجیرهٔ interceptorها

```
call ─▶ application interceptors   once per call
        RetryAndFollowUp            redirects · retries
        Bridge                      gzip · cookies · headers
        Cache                       may answer from disk
        Connect                     pool · TLS
     ─▶ network interceptors        once per network hop
        CallServer                  bytes on the wire
```

	`addInterceptor`	`addNetworkInterceptor`
کِی صدا زده می‌شود	یک بار برای هر call، حتی وقتی از cache جواب می‌آید	برای هر درخواست شبکه: هر redirect و هر retry؛ نه برای cache
چه می‌بیند	درخواستی که کد شما ساخته	درخواست نهایی (`Host`، `Accept-Encoding: gzip`) و خود connection
`proceed()`	صفر بار (قطع زنجیره) یا چند بار	دقیقاً یک بار
مناسب برای	هدر auth، لاگ، سیاست سطح اپ	متریک شبکه، دیباگ سطح سیم



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. یک interceptor برای هدر auth

```
class AuthHeaderInterceptor(
    private val tokens: TokenStore,
) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val token = tokens.accessToken()
            ?: return chain.proceed(chain.request())
        val request = chain.request().newBuilder()
            .header("Authorization", "Bearer $token")
            .build()
        return chain.proceed(request)
    }
}
```

- interceptorها روی تردهای OkHttp اجرا می‌شوند و **blocking** هستند — سریع نگهشان دارید
- `header()` هدر را جایگزین می‌کند؛ `addHeader()` نسخهٔ تکراری اضافه می‌کند

تلهٔ مصاحبه

`HttpLoggingInterceptor` با سطح `BODY` در release، توکن و دادهٔ شخصی را در Logcat می‌نویسد. در release از `NONE` و در debug از `redactHeader("Authorization")` استفاده کنید.

تلهٔ مصاحبه

اسلایدهای پارسال `CertificatePinner` را network interceptor و `Authenticator` را interceptor معرفی کرده بودند. هیچ‌کدام نیستند: با `certificatePinner()` و `authenticator()` تنظیم می‌شوند (روز ۱۲).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. timeoutها: هر کدام واقعاً چه چیزی را پوشش می‌دهد

timeout در OkHttp	پیش‌فرض	پوشش
`connectTimeout`	۱۰ ثانیه	باز کردن اتصال TCP به host
`readTimeout`	۱۰ ثانیه	بیشترین **فاصله** بین بایت‌ها هنگام خواندن — نه کل زمان
`writeTimeout`	۱۰ ثانیه	بیشترین فاصله بین نوشتن‌های درخواست
`callTimeout`	ندارد	**کل** فراخوانی: DNS، اتصال، ارسال، زمان سرور، body، redirectها و retryها

- پاسخی که هر ۹ ثانیه یک بایت می‌فرستد هیچ‌وقت `readTimeout` را فعال نمی‌کند — فقط `callTimeout` زمان انتظار کاربر را محدود می‌کند
- در coroutineها `withTimeout { api.user(id) }` هم کار می‌کند: لغو coroutine فراخوانی OkHttp را هم لغو می‌کند

تلهٔ مصاحبه

«*timeoutها جلوی یخ زدن ترد UI را می‌گیرند.*» — ادعای پارسال. فراخوانی شبکه هیچ‌وقت روی ترد اصلی اجرا نمی‌شود (اندروید `NetworkOnMainThreadException` پرتاب می‌کند). timeout فقط مدت انتظار یک فراخوانی **پس‌زمینه** — و کاربر — را محدود می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. retry با exponential backoff و jitter

```kotlin
suspend fun <T> withRetry(
    maxAttempts: Int = 3,
    baseDelayMs: Long = 500,
    maxDelayMs: Long = 8_000,
    isTransient: (Throwable) -> Boolean,
    block: suspend () -> T,
): T {
    var cap = baseDelayMs
    repeat(maxAttempts - 1) {
        try {
            return block()
        } catch (e: CancellationException) {
            throw e
        } catch (e: Exception) {
            if (!isTransient(e)) throw e
        }
        delay(Random.nextLong(cap + 1)) // full jitter
        cap = (cap * 2).coerceAtMost(maxDelayMs)
    }
    return block()
}
```

- فقط خطاهای **گذرا** را retry کنید: `IOException`، `408`، `429` و `5xx` — آن هم فقط برای فراخوانی‌های **idempotent**
- jitter کلاینت‌ها را پخش می‌کند تا هزاران گوشی در یک ثانیه با هم retry نکنند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. OkHttp خودش چه چیزی را retry می‌کند

تلهٔ مصاحبه

«*OkHttp خطاهای گذرا مثل مشکل DNS را خودکار retry می‌کند.*» — نه به آن معنایی که اسلایدهای پارسال می‌گفتند. گزینهٔ `retryOnConnectionFailure` که پیش‌فرض روشن است، فقط بی‌صدا از چند مشکل **اتصال** عبور می‌کند — یک connection کهنه در pool، یا مسیری که در دسترس نیست وقتی IP دیگری هم هست. هیچ‌وقت `503` را retry نمی‌کند و backoff هم ندارد.

تلهٔ مصاحبه

retry کردن `POST /payments` بعد از timeout. شاید سرور کارت را شارژ کرده و فقط پاسخ گم شده باشد. فراخوانی‌های غیر idempotent را فقط با یک **idempotency key** دوباره بفرستید (سؤال ۱۲).

سؤال تکمیلی

«*سرور هدر `Retry-After` فرستاده. حالا چه؟*» — این هدر بر فرمول backoff شما مقدم است. مقدارش می‌تواند ثانیه یا یک تاریخ HTTP باشد؛ هر دو را پشتیبانی کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. خطا را یک بار، در مرز لایه مدل کنید

```
suspend fun <T> apiCall(block: suspend () -> T): ApiResult<T> =
    try {
        ApiResult.Ok(block())
    } catch (e: CancellationException) {
        throw e
    } catch (e: SocketTimeoutException) {
        ApiResult.Err(ApiError.Timeout)
    } catch (e: IOException) {
        ApiResult.Err(ApiError.Network)
    } catch (e: HttpException) {
        val body = e.response()?.errorBody()?.string()
        ApiResult.Err(ApiError.Http(e.code(), body))
    } catch (e: SerializationException) {
        ApiResult.Err(ApiError.Parsing(e))
    }
```

- ترتیب مهم است: `SocketTimeoutException` خودش یک `IOException` **است** — اول آن را بگیرید
- `ApiError` یک sealed interface است؛ ViewModel آن را به پیام تبدیل می‌کند و هیچ‌وقت `HttpException` نمی‌بیند

تلهٔ مصاحبه

دسته‌بندی خطا با گشتن دنبال یک کلمه در متن `e.message`، همان‌طور که اسلایدهای پارسال پیشنهاد می‌کردند. متن پیام‌ها بین نسخه‌ها و زبان‌ها عوض می‌شود — از **نوع** exception و status code استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. از خطا تا تجربهٔ کاربر

خطا	کاربر چه می‌بیند	retry؟
شبکه (`UnknownHostException`، `ConnectException`)	«آفلاین هستید»	بله — وقتی شبکه برگشت
timeout	«کمی بیشتر از معمول طول می‌کشد»	بله، اگر idempotent باشد
`401`	هیچ — refresh بی‌صدا (روز ۱۲)	یک بار، بعد از refresh
`400`، `404`، `422`	پیام مشخص از body خطا	خیر
`429`، `503`	«شلوغ است، دوباره تلاش می‌کنیم...»	بله، بعد از `Retry-After`
بقیهٔ `5xx`	«مشکلی سمت ما پیش آمده»	بله، با backoff
خطای parse	پیام عمومی؛ ثبت یک non-fatal	خیر — این یک باگ است

تلهٔ مصاحبه

«*CancellationException هم یک خطای معمولی است — لاگش کنید.*» — توصیهٔ پارسال. این اصلاً شکست **نیست**: دوباره throw کنید، وگرنه لغو ساختاریافته خراب می‌شود. `runCatching` دور فراخوانی‌های suspend هم همین مشکل را دارد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. HTTP caching: هدرهایی که تصمیم می‌گیرند

هدر	معنی
`Cache-Control: max-age=60`	تا ۶۰ ثانیه تازه است — از دیسک، بدون شبکه
`Cache-Control: no-cache`	ذخیره می‌شود، اما قبل از هر استفاده باید **revalidate** شود
`Cache-Control: no-store`	هرگز روی دیسک نوشته نشود
`ETag` و سپس `If-None-Match`	revalidate بر اساس نسخه؛ `304` یعنی «نسخهٔ شما هنوز درست است»
`Last-Modified` و سپس `If-Modified-Since`	revalidate بر اساس تاریخ

- وقتی یک `Cache` به OkHttp بدهید، همهٔ این‌ها را خودش پیاده می‌کند — از جمله درخواست شرطی و `304`
- OkHttp فقط پاسخ‌های `GET` را cache می‌کند

تلهٔ مصاحبه

«`no-cache` یعنی *cache نکن*.» — آن `no-store` است. `no-cache` پاسخ را ذخیره می‌کند ولی اول با سرور چک می‌کند — که وقتی جواب یک `304` بدون body باشد خیلی ارزان است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. cache در کد — و محدودیت‌هایش

```
val cacheDir = File(context.cacheDir, "http")
val client = OkHttpClient.Builder()
    .cache(Cache(cacheDir, 20L * 1024 * 1024)) // 20 MiB
    .build()

interface CatalogApi {
    @GET("catalog")
    suspend fun catalog(): CatalogDto

    // Offline fallback: cached copy or HTTP 504 — never network
    @GET("catalog")
    @Headers("Cache-Control: only-if-cached, max-stale=604800")
    suspend fun cachedCatalog(): CatalogDto
}
```

تلهٔ مصاحبه

یک network interceptor که هر پاسخی را به `max-age=3600` بازنویسی می‌کند «تا caching کار کند». قواعد سرور را زیر پا می‌گذارید و ممکن است دادهٔ شخصی cache شود. هدرها را روی سرور درست کنید.

سؤال تکمیلی

«HTTP cache یا Room؟» — HTTP cache شفاف است اما مات: بایت‌هایی که ممکن است پاک شوند و نمی‌توانید query یا observe کنید. برای صفحه‌های offline-first، **دیتابیس** منبع واحد حقیقت است (روزهای ۱۳ و ۱۴).



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

Ktor و KMP، وضعیت اتصال، لغو و هم‌روندی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. کلاینت Ktor: یک لایهٔ شبکه برای KMP

```
val client = HttpClient(OkHttp) { // Darwin engine on iOS
    expectSuccess = true
    install(ContentNegotiation) { json(json) }
    install(HttpTimeout) { requestTimeoutMillis = 15_000 }
    install(HttpRequestRetry) {
        retryOnServerErrors(maxRetries = 3)
        exponentialDelay()
    }
    defaultRequest { url("https://api.example.com/v1/") }
}

suspend fun user(id: String): UserDto =
    client.get("users/$id").body()
```

- پلاگین‌ها جای interceptorها را می‌گیرند: `Auth`، `HttpCache`، `Logging`، `HttpTimeout` و `HttpRequestRetry`
- engine برای هر پلتفرم: روی اندروید OkHttp یا Android، روی iOS موتور Darwin، روی JVM موتور CIO



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. Retrofit یا Ktor؟

	Retrofit و OkHttp	کلاینت Ktor
سبک	اینترفیس با annotation، فراخوانی تولیدشده	تابع‌های suspend ساده با یک DSL
پلتفرم‌ها	اندروید و JVM	Kotlin Multiplatform
توسعه‌پذیری	interceptor، converter، call adapter	پلاگین
بهترین انتخاب وقتی	اپ فقط اندرویدی، کدبیس موجود	لایهٔ data مشترک با iOS

تلهٔ مصاحبه

«*Ktor روی 404 خطا پرتاب می‌کند.*» — فقط اگر خودتان بخواهید. از Ktor 2 به بعد، مقدار پیش‌فرض `expectSuccess` برابر `false` است: پاسخ را می‌گیرید و باید `status` را چک کنید. با `expectSuccess = true` خطای `ClientRequestException` یا `ServerResponseException` می‌گیرید.

سؤال تکمیلی

«*هر کدام را چطور تست می‌کنید؟*» — با پاسخ‌های HTTP واقعی، نه mock کردن اینترفیس: برای Ktor از `MockEngine` و برای OkHttp از `MockWebServer`. نگاشت خطاهایتان را با یک `503`، یک JSON خراب و یک body کند تست کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. مشاهدهٔ درست وضعیت اتصال

```
private fun NetworkCapabilities?.validated() =
    this?.hasCapability(NET_CAPABILITY_VALIDATED) == true

fun ConnectivityManager.onlineFlow() = callbackFlow<Boolean> {
    val callback = object : NetworkCallback() {
        override fun onCapabilitiesChanged(
            network: Network,
            caps: NetworkCapabilities,
        ) {
            trySend(caps.validated())
        }
        override fun onLost(network: Network) {
            trySend(false)
        }
    }
    trySend(getNetworkCapabilities(activeNetwork).validated())
    registerDefaultNetworkCallback(callback)
    awaitClose { unregisterNetworkCallback(callback) }
}.distinctUntilChanged()
```

- `VALIDATED` یعنی سیستم واقعاً به اینترنت رسیده — نه فقط «Wi-Fi روشن است» (مثل captive portal)
- `activeNetworkInfo` و `NetworkInfo` از API 29 منسوخ (deprecated) شده‌اند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. قواعد وضعیت اتصال

- از این flow برای **راهنمایی در UI** («آفلاین هستید») و برای refresh خودکار بعد از برگشتن شبکه استفاده کنید
- آپلود و همگام‌سازی قابل تعویق: WorkManager با قید شبکه (روز ۱۵)
- به شبکهٔ حجمی (metered) احترام بگذارید: قبل از دانلود بزرگ `NET_CAPABILITY_NOT_METERED` را چک کنید

تلهٔ مصاحبه

«*قبل از هر درخواست* `isConnected` *را چک کن.*» — شبکه ممکن است بین چک و فراخوانی عوض شود، و «وصل» به معنی «اینترنت دارد» نیست. **خود درخواست را بفرستید** و شکست را مدیریت کنید؛ وضعیت اتصال یک راهنماست، نه یک دروازه.

سؤال تکمیلی

«*کاربر آفلاین است و روی ارسال می‌زند. چه می‌شود؟*» — اول محلی ذخیره کنید، به‌صورت «در انتظار» نشانش دهید، و بگذارید WorkManager وقتی شبکه برگشت آن را بفرستد. ورودی کاربر هرگز نباید به خاطر شبکه گم شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. لغو، main-safety و هم‌روندی

```
suspend fun loadProfile(id: String): Profile = coroutineScope {
    val user = async { api.user(id) }
    val stats = async { api.stats(id) }
    Profile(user.await().toDomain(), stats.await().toDomain())
}
```

- فراخوانی‌های `suspend` در Retrofit و Ktor **main-safe** هستند؛ لغو coroutine، **فراخوانی HTTP را هم لغو می‌کند**
- با `coroutineScope` اگر یک درخواست شکست بخورد، دیگری لغو می‌شود — هیچ کاری نشت نمی‌کند
- کاربر از صفحه خارج می‌شود، `viewModelScope` لغو می‌شود و خواندن‌های در جریان متوقف می‌شوند

تلهٔ مصاحبه

«`awaitAll` روی ۵۰ فراخوانی به یک host، ۵۰ درخواست موازی اجرا می‌کند.» — `Dispatcher` در OkHttp به‌طور پیش‌فرض فقط **۵ فراخوانی async هم‌زمان برای هر host** (و ۶۴ در کل) اجازه می‌دهد؛ بقیه صف می‌شوند. یک endpoint دسته‌ای (batch) بهتر از دست بردن در `maxRequestsPerHost` است.

تلهٔ مصاحبه

ارسال پیام یا پرداخت از `viewModelScope`: کاربر از صفحه خارج می‌شود و عملیات نوشتن وسط کار لغو می‌شود. نوشتن‌هایی که باید تمام شوند جایشان در یک coroutine با scope سطح اپ یا WorkManager است.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

طراحی ماژول شبکه، مقاومت در مقیاس بالا، تکامل API



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. طراحی ماژول شبکه

```
:core:network     one OkHttpClient · Json · interceptors
        ▲  newBuilder() per backend
:feature:x:data   XApi · XRemoteDataSource · DTO → model
        ▲  domain models only
:feature:x:ui     ViewModel · screens
```

```
val base = OkHttpClient.Builder()
    .cache(cache)
    .addInterceptor(userAgentInterceptor)
    .build()

val paymentsClient = base.newBuilder()
    .callTimeout(30, TimeUnit.SECONDS)
    .addInterceptor(PaymentsAuthInterceptor(tokens))
    .build()
```

- `newBuilder()` connection pool، dispatcher و cache را **به اشتراک می‌گذارد** — سیاست متفاوت، منابع مشترک
- محیط‌ها (environment) از build type یا flavor می‌آیند، نه از عوض کردن base URL در زمان اجرا در نسخهٔ release



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. قواعد ماژول

- یک `Json` و یک `OkHttpClient` ریشه که DI فراهم می‌کند؛ هر بک‌اند client مشتق‌شده و اینترفیس API خودش را دارد
- data sourceهای remote مدل domain یا خطای تایپ‌دار برمی‌گردانند — `HttpException` هرگز از لایهٔ data بیرون نمی‌رود
- دغدغه‌های مشترک فقط یک جا در `:core:network` زندگی می‌کنند: user agent، auth، لاگ، متریک

تلهٔ مصاحبه

هر فیچر `OkHttpClient` خودش را از صفر می‌سازد: N تا connection pool، N تا thread pool — و دو نمونهٔ `Cache` روی یک پوشه همدیگر را خراب می‌کنند. client را مشتق کنید، تکثیر نکنید.

سؤال تکمیلی

«*در production چطور می‌فهمید چه چیزی کند است؟*» — یک `EventListener` در OkHttp زمان DNS، اتصال، TLS و پاسخ را برای هر فراخوانی گزارش می‌دهد؛ آن را به سیستم مانیتورینگ بدهید (روز ۲۸).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. مقاومت در مقیاس بالا

الگو	از چه چیزی محافظت می‌کند
timeout روی هر فراخوانی	گیر کردن بی‌پایان روی یک اتصال مرده
backoff و jitter با سقف تلاش	طوفان retry بعد از یک قطعی
احترام به `Retry-After` روی `429` و `503`	کوبیدن سروری که خودش مهلت خواسته
idempotency key	شارژ دوباره وقتی retry بعد از گم شدن پاسخ می‌آید
حذف درخواست‌های تکراری	پنج صفحه که هم‌زمان یک پروفایل را می‌گیرند
circuit breaker یا kill switch	صدا زدن endpointی که می‌دانیم از کار افتاده
کاهش تدریجی قابلیت‌ها	یک ویجت خراب که کل صفحه را پایین می‌آورد

تلهٔ مصاحبه

retry در همهٔ لایه‌ها: ۳ تلاش در اپ، ضربدر ۳ در gateway، ضربدر ۳ در سرویس = تا **۲۷** درخواست برای یک لمس. تصمیم بگیرید فقط یک لایه مسئول retry باشد، و یک بودجهٔ retry داشته باشید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. idempotency key در عمل

```
interface PaymentsApi {
    @POST("payments")
    suspend fun pay(
        @Header("Idempotency-Key") key: String,
        @Body body: PaymentDto,
    ): PaymentResultDto
}

// One key per user intent — reused on every retry
val key = UUID.randomUUID().toString()
val result = withRetry(isTransient = ::isTransient) {
    api.pay(key, payment)
}
```

- سرور کلید را همراه نتیجه ذخیره می‌کند؛ retry با همان کلید **همان** نتیجه را برمی‌گرداند به‌جای شارژ دوباره
- کلید را همراه عملیات در انتظار ذخیره کنید تا از process death و retryهای WorkManager عبور کند

نظر من

retry یک تصمیم محصولی است، نه جزئیات شبکه. برای هر فراخوانی بنویسید: idempotent است؟ چه کسی retry می‌کند؟ کاربر در این مدت چه می‌بیند؟



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. تکامل API بدون خراب کردن اپ‌های قدیمی

نسخه‌های قدیمی اپ سال‌ها نصب می‌مانند. سرور هرگز نباید آن‌ها را خراب کند، و کلاینت باید تغییر را تحمل کند.

```
@Serializable
enum class Status { ACTIVE, SUSPENDED, UNKNOWN }

@Serializable
data class AccountDto(
    val id: String,
    // New server values → UNKNOWN (with coerceInputValues)
    val status: Status = Status.UNKNOWN,
    // Added in a later API version: optional
    val nickname: String? = null,
)
```

تلهٔ مصاحبه

سرور یک مقدار جدید به enum اضافه می‌کند و همهٔ اپ‌های قدیمی موقع parse کرش می‌کنند. همیشه یک راه برگشت داشته باشید: پیش‌فرض `UNKNOWN` برای enumها، یک deserializer پیش‌فرض برای نوع‌های polymorphic، و `ignoreUnknownKeys` برای objectها.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. قرارداد سازگاری

تغییر	امن است؟
اضافه کردن فیلد اختیاری یا endpoint جدید	بله — کلاینت‌های قدیمی نادیده‌اش می‌گیرند
اضافه کردن مقدار به enum	فقط اگر کلاینت‌ها راه برگشت داشته باشند
تغییر نام یا حذف فیلد	خیر — فیلد جدید اضافه کنید، قدیمی را deprecate کنید، بعد از مهلت حذف کنید
تغییر نوع یا معنای یک فیلد	خیر — فیلد جدید یا نسخهٔ جدید API

- نسخه را جای قابل‌دیدن بگذارید: `/v2/` در مسیر یا یک هدر نسخه — یک روش، همه‌جا یکسان
- یک endpoint برای **حداقل نسخهٔ پشتیبانی‌شده**، به‌علاوهٔ in-app update برای تغییرات شکنندهٔ نادر (روز ۲۴)
- تست قرارداد (contract test) یا کلاینت تولیدشده از OpenAPI، شکستن را در CI پیدا می‌کند، نه در production

جواب سینیور در یک جمله

«فرض کن هر فراخوانی می‌تواند کند باشد، شکست بخورد یا دو بار اجرا شود: timeout محدود، retry فقط جایی که امن است، خطای تایپ‌دار در مرز لایه، و قرارداد API که تغییر را تحمل کند.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۱۱ — خلاصهٔ یک‌صفحه‌ای

- `DELETE` و `PUT` idempotent هستند، `POST` نه؛ idempotent با امن فرق دارد.
- روی `401` توکن را refresh کنید، روی `403` نه؛ `204` body ندارد.
- Retrofit توصیف می‌کند، OkHttp اجرا؛ `suspend` از 2.6 بومی است.
- kotlinx.serialization با `ignoreUnknownKeys`؛ Gson پیش‌فرض‌های Kotlin را نادیده می‌گیرد.
- application interceptor یک بار در هر call؛ network interceptor در هر رفت‌وبرگشت.
- `readTimeout` فاصله است نه کل زمان؛ کل را با `callTimeout` محدود کنید.
- retry فقط برای خطای گذرا و idempotent، با backoff و jitter؛ برای نوشتن idempotency key.
- خطا را در لایهٔ data به `ApiError` تبدیل کنید؛ `CancellationException` را دوباره throw کنید.
- `no-cache` با `no-store` فرق دارد؛ منبع حقیقت offline-first دیتابیس Room است.
- وضعیت اتصال راهنماست، نه دروازه؛ یک `OkHttpClient` ریشه، بقیه با `newBuilder()`.

روز ۱۲ در راه است: لایهٔ شبکه، بخش ۲ — امنیت و مقیاس. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700